

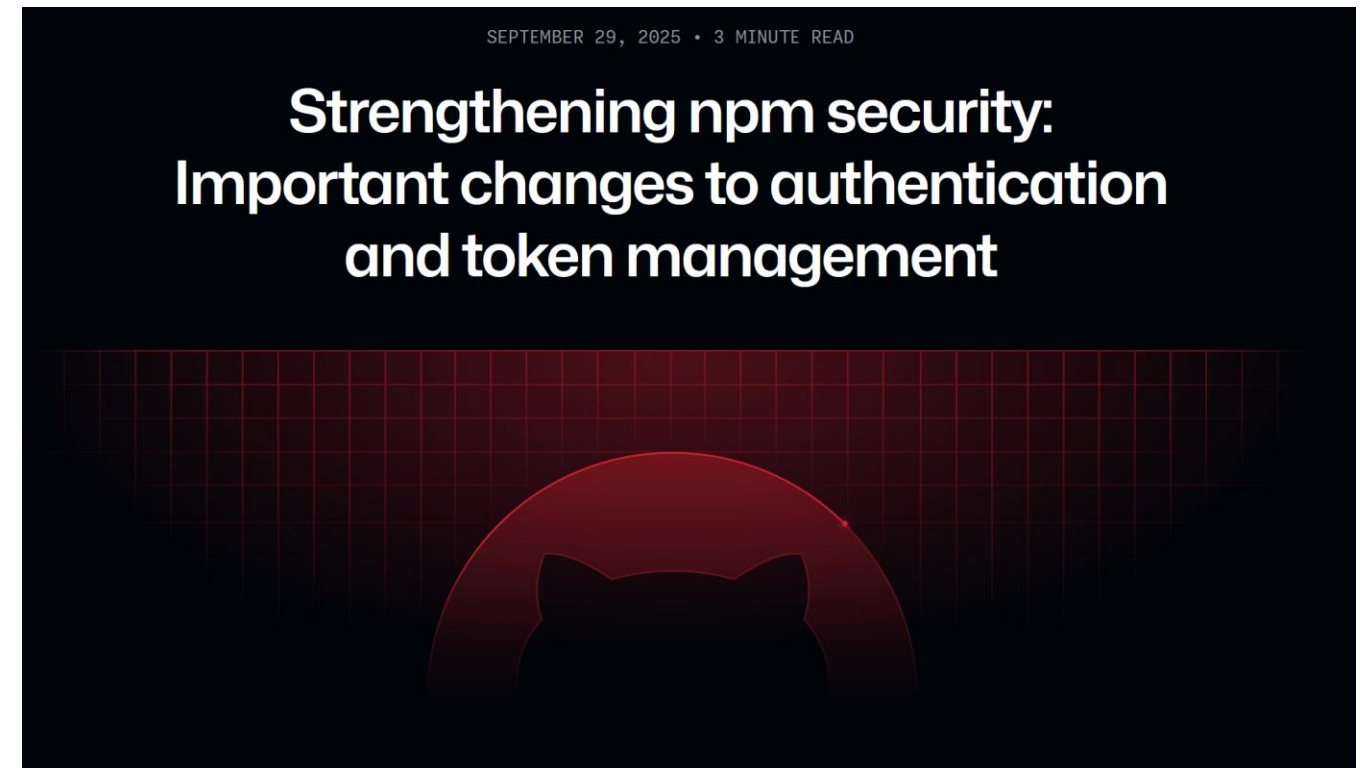
Developer Operations (devops)

Continuous Integration / -deployment



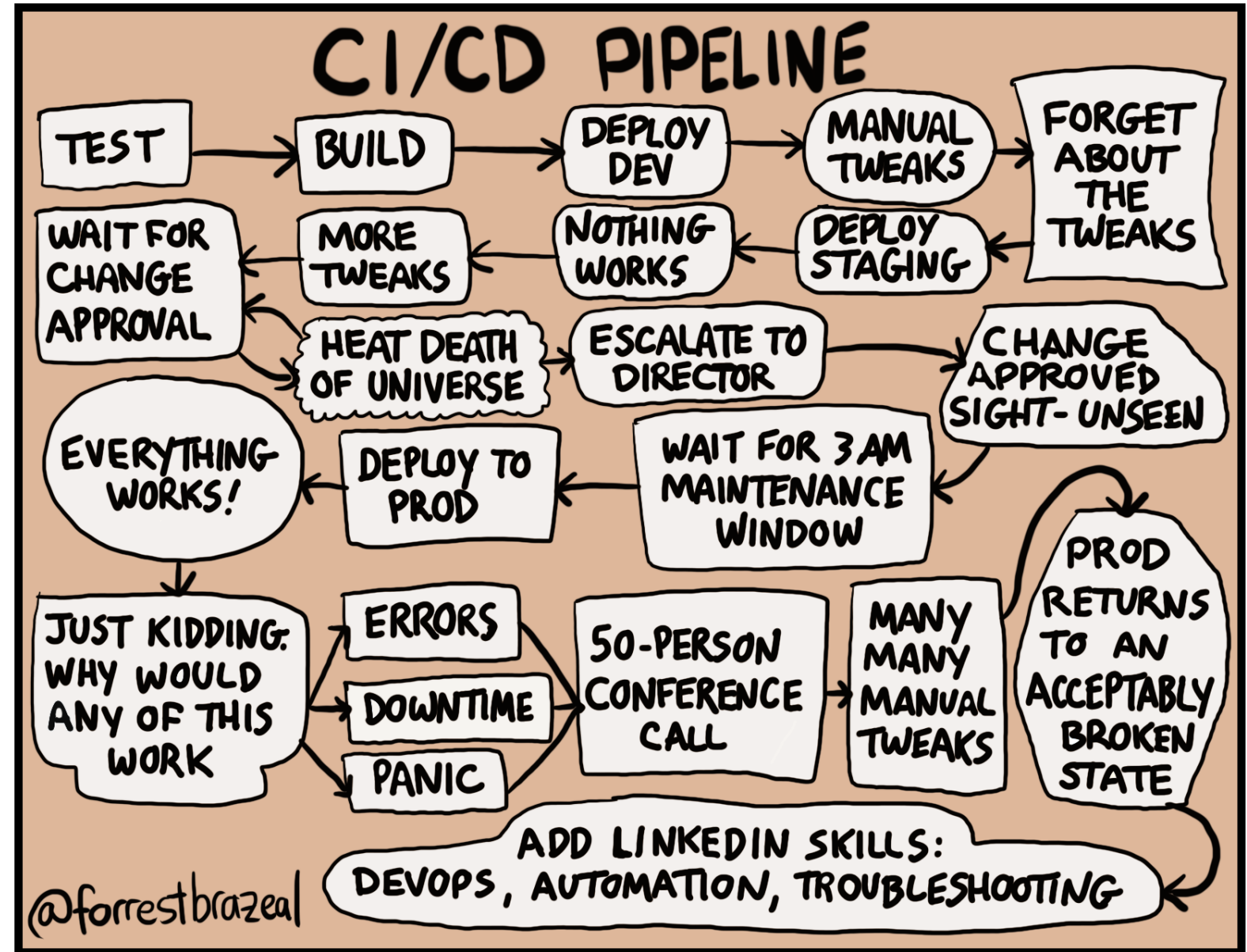
Breaking News - Strengthening npm security

- **Npm access token lifetime limits**
 - A default expiration of seven days, reduced from 30 days.
 - A maximum expiration of 90 days, which used to be unlimited.
- **TOTP retired as 2FA**
 - Passkeys will be used in the future
- **Push towards “trusted publishing (OIDC)”**
 - Currently only supported by GitLab and GitHub



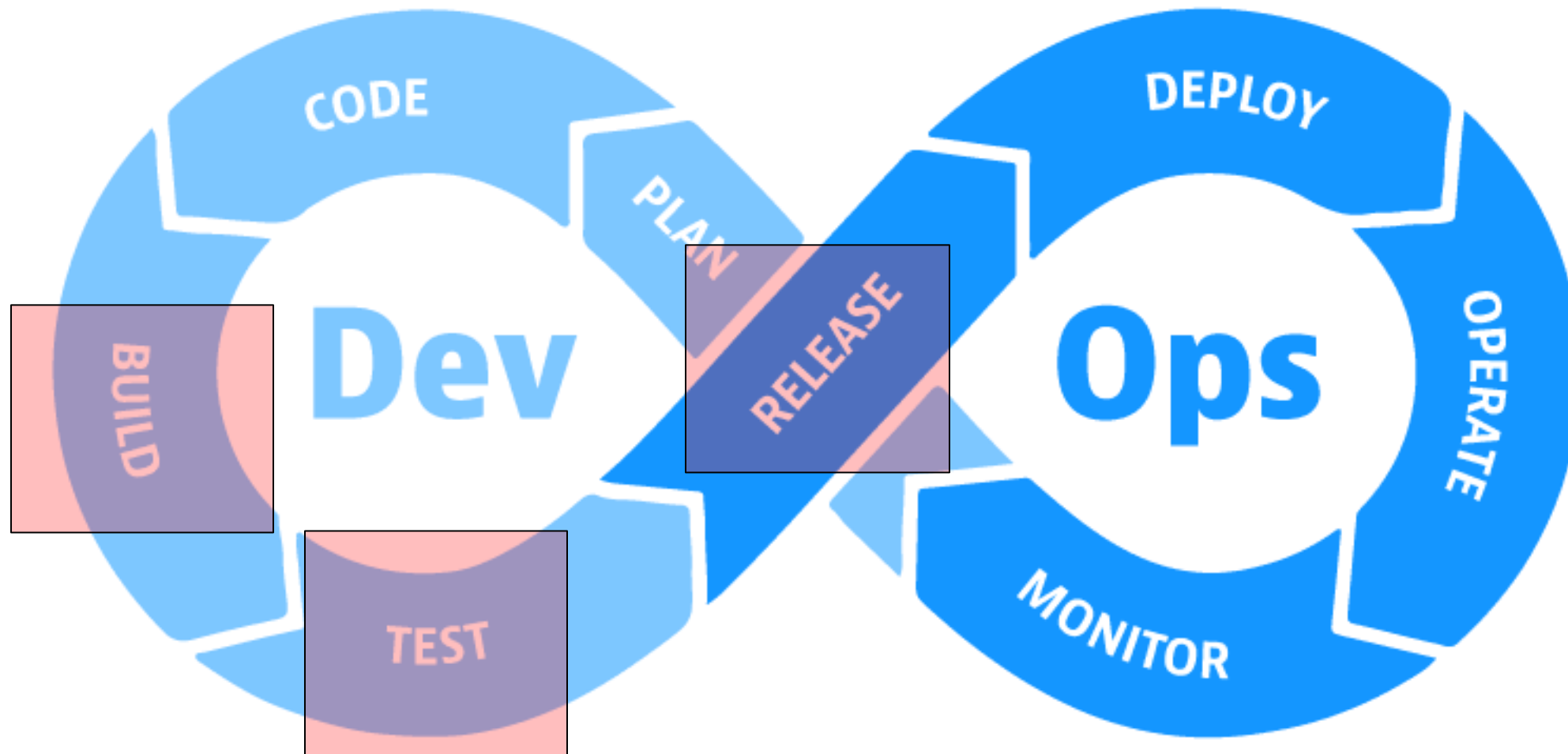
<https://github.blog/changelog/2025-09-29-strengthening-npm-security-important-changes-to-authentication-and-token-management/>

Building Software and deploying it



© 2018 Forrest Brazeal. All rights reserved.

Build and Release

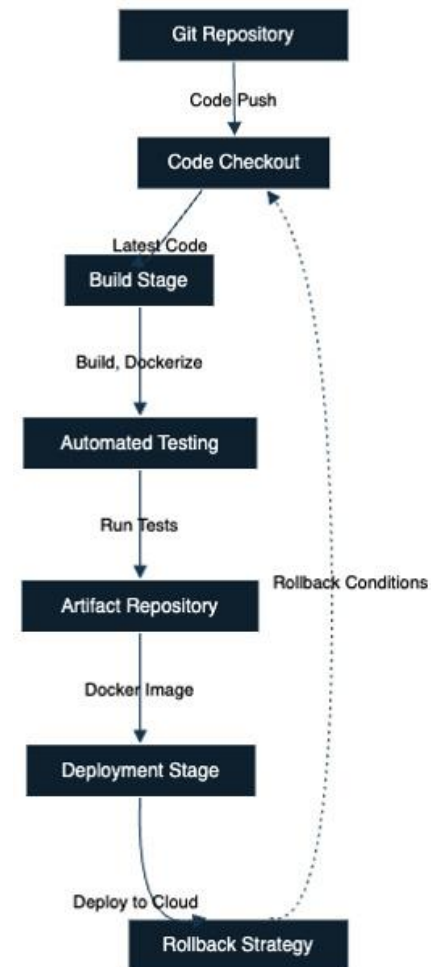


Build and Deploy-Pattern

Separate Build and Deploy

- Increases stability and robustness
- One artifact for testing and production (stage parity in 12factor methodology)
- Should base on release-workflow of source code management

- Focus on Build for today



Example build and deploy model

Application Structure:

1. Source Code Repository:
 - Hosted on Git.
2. Web Application:
 - A simple Node.js application with an `app.js` file

CI/CD Pipeline:

1. Code Checkout:
 - Triggered on each code push to the GitHub repository
 - Checks out the latest code
2. Build Stage:
 - Tasks:
 - Installs Node.js dependencies
 - Builds the application
 - Creates a Docker image
3. Automated Testing:
 - Tasks:
 - Runs unit tests using a testing framework (for example, Mocha)
4. Artifact Repository:
 - Pushes the Docker image to a container registry (for example, Docker Hub)
5. Deployment Stage:
 - Tasks:
 - Deploys the Docker image to a cloud platform (for example, AWS, Azure, or Google Cloud)
 - May involve creating or updating infrastructure using Infrastructure as Code (IaC) tools
6. Rollback Strategy:
 - Defines conditions for rolling back to a previous version in case of deployment failures



Why to use a Build System?

Independent Platform:

- Single Source of Truth → Source Control
- Single Source of Build → Build System

Defines:

- Build Tool Version
- Guardrails regarding testing / analysis
- Transparency to build process



Why to use a Build System?

Centralized Credential Handling:

Access to all kinds of necessary adjacent toolsets such as:

- Repositories / Registries
- Analysis Platforms (Sonarqube)
- Issue Management
- Additional Testing frameworks
- (Platforms to be provisioned)



Why to use a Build System?

Independent Build Status:

Transparent Status about the build:

- Alignment of Team
- Ability to release (if necessary)
- Status of Tests
- Prioritization of issues
- “Operating Status of the Source”



Why to use a Build System?

Increasing the bus factor (also known as the “truck number”):

“How many or few would have to be hit by a truck (or quit) before the project is incapacitated?” – JIM COPLIEN

Bus Factor == 1 → immediate time for action

Modern CI-Systems



Travis CI



GitLab



Jenkins



GitHub Actions



circleci

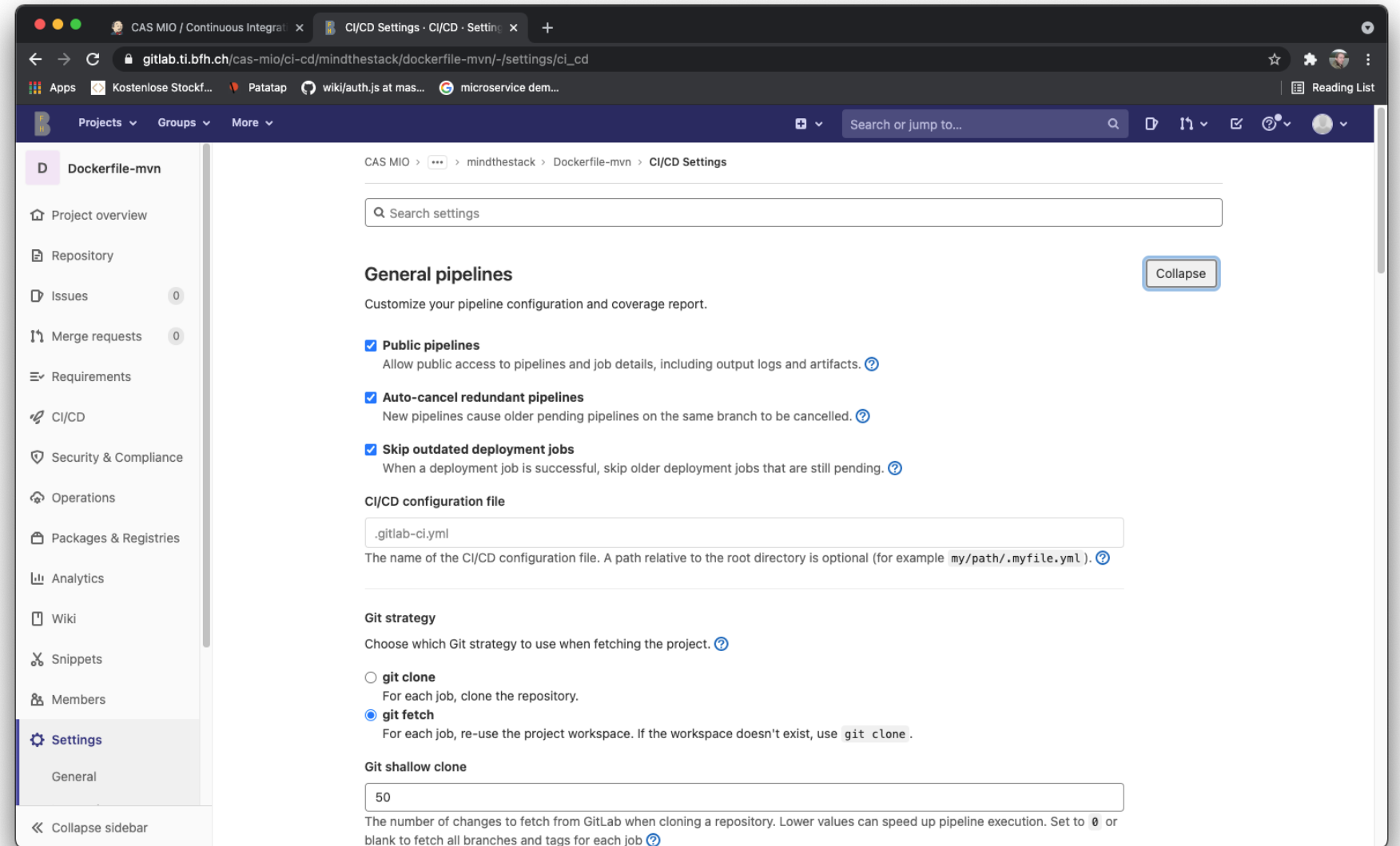


Bamboo

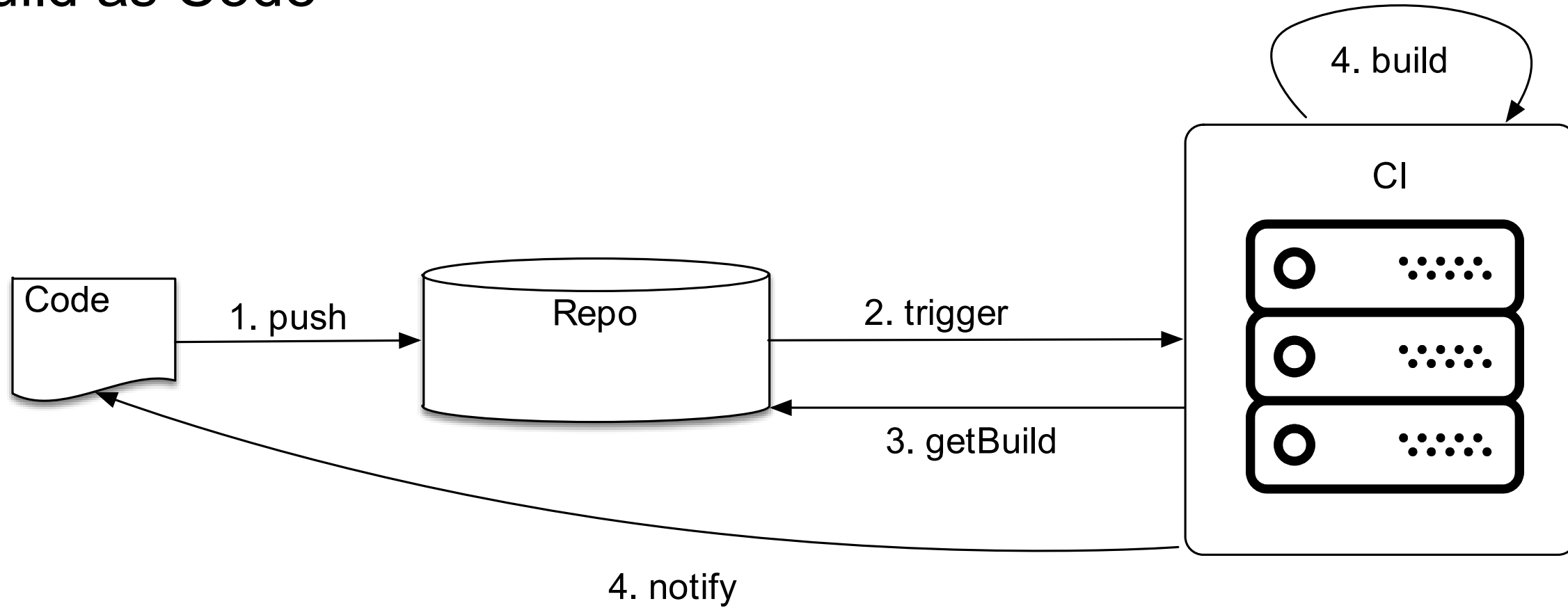


Build as Code, Gitlab

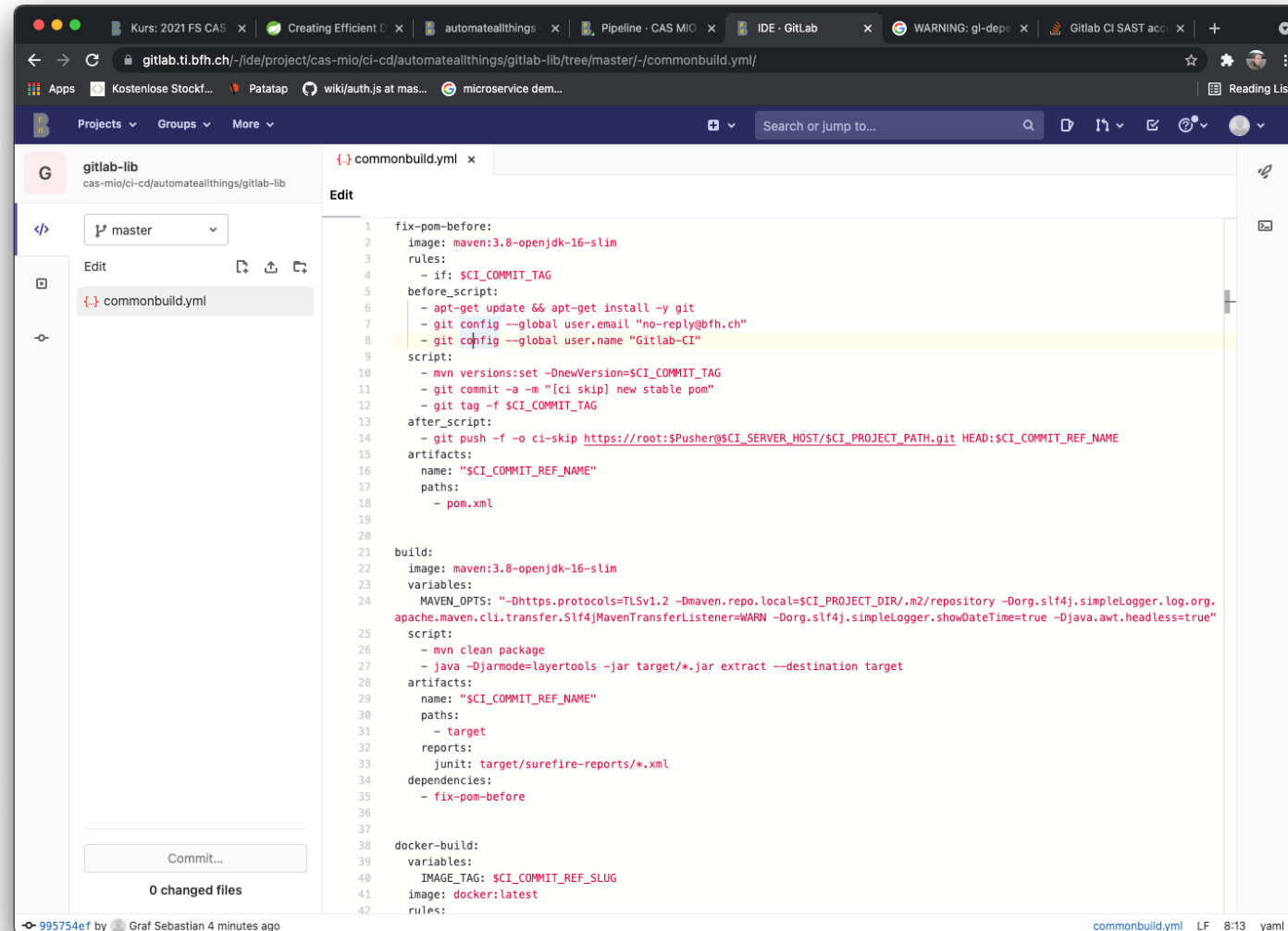
DEMO



Build as Code



Gitlab Web IDE



```

1 fix-pom-before:
2   image: maven:3.8-openjdk-16-slim
3   rules:
4     - if: $CI_COMMIT_TAG
5   before_script:
6     - apt-get update && apt-get install -y git
7     - git config --global user.email "no-reply@bfh.ch"
8     - git config --global user.name "Gitlab-CI"
9   script:
10    - mvn versions:set -DnewVersion=$CI_COMMIT_TAG
11    - git commit -a -m "[ci skip] new stable pom"
12    - git tag -f $CI_COMMIT_TAG
13  after_script:
14    - git push -f -o ci-skip https://root:$Pusher@$CI_SERVER_HOST/$CI_PROJECT_PATH.git HEAD:$CI_COMMIT_REF_NAME
15  artifacts:
16    name: "$CI_COMMIT_REF_NAME"
17    paths:
18      - pom.xml
19
20
21  build:
22    image: maven:3.8-openjdk-16-slim
23    variables:
24      MAVEN_OPTS: "-Dhttps.protocols=TLSv1.2 -Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository -Dorg.slf4j.simpleLogger.log.org.apache.maven.cli.transfer.Slf4jMavenTransferListener=WARN -Dorg.slf4j.simpleLogger.showDateTime=true -Djava.awt.headless=true"
25    script:
26      - mvn clean package
27      - java -DjarMode=layertools -jar target/*.jar extract --destination target
28    artifacts:
29      name: "$CI_COMMIT_REF_NAME"
30      paths:
31        - target
32      reports:
33        junit: target/surefire-reports/*.xml
34    dependencies:
35      - fix-pom-before
36
37
38  docker-build:
39    variables:
40      IMAGE_TAG: $CI_COMMIT_REF_SLUG
41    image: docker:latest
42    rules:
  
```

DEMO

Gitlab-CI Development



gitlab.ti.bfh.ch/cas-mio/ci-cd/automateallthings/gitlab-ci/-/ci/editor

Projects Groups More

Search or jump to...

CAS MIO > automateallthings > Gitlabci > Pipeline Editor

Pipeline #93155 passed for 798921cc

✓ This GitLab CI configuration is valid. [Learn more](#)

Write pipeline configuration Visualize Lint View merged YAML

```

1 include:
2   - project: 'cas-mio/ci-cd/automateallthings/gitlab-lib'
3     ref: master
4     file: 'commonbuild.yml'
5
6 fix-pom-before:
7   stage: .pre
8
9 build:
10  stage: build
11
12 docker-build:
13   stage: deploy
14
15

```

Commit message

Update .gitlab-ci.yml file

<https://gitlab.ti.bfh.ch/cas-mio/ci-cd/automateallthings/gitlab-ci/-/ci/editor#>

gitlab.ti.bfh.ch/cas-mio/ci-cd/automateallthings/gitlab-ci/-/ci/editor

Projects Groups More

Search or jump to...

CAS MIO > automateallthings > Gitlabci > Pipeline Editor

Pipeline #93155 passed for 798921cc

✓ This GitLab CI configuration is valid. [Learn more](#)

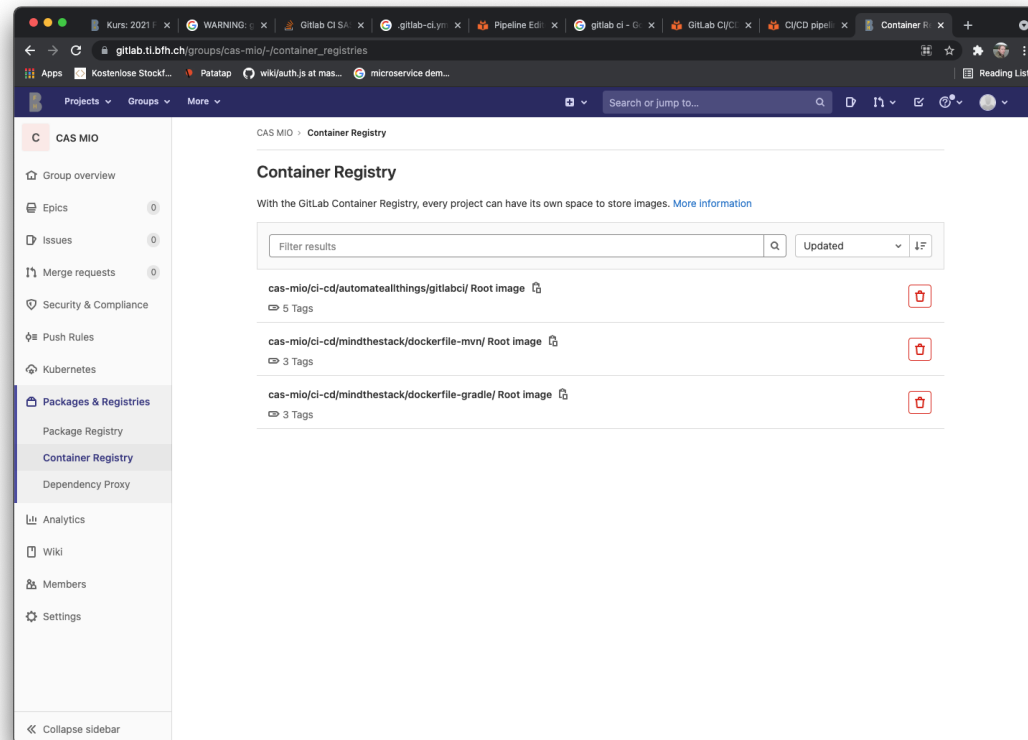
Write pipeline configuration Visualize Lint View merged YAML

Status:
Syntax is correct. CI configuration validated, including all configuration added with the includes keyword. [More information](#)

Parameter	Value
	<pre> apt-get update && apt-get install -y git git config --global user.email "no-reply@bfh.ch" git config --global user.name "Gitlab-CI" </pre>
.pre Job - fix-pom-before	<pre> mvn versions:set -DnewVersion=\${CI_COMMIT_TAG} git commit -a -m "[ci skip] new stable pom" git tag -f \${CI_COMMIT_TAG} git push -f -o ci-skip https://root:\$Pusher@\${CI_SERVER_HOST}/\${CI_PROJECT_PATH}.git HEAD:\${CI_COMMIT_REF_NAME} </pre> When: on_success
Build Job - build	<pre> mvn clean package java -Djarmode=layertools -jar target/*.jar extract --destination target </pre> Only policy: branches, tags When: on_success
Deploy Job - docker-build	<pre> docker login -u "\${CI_REGISTRY_USER}" -p "\${CI_REGISTRY_PASSWORD}" \${CI_REGISTRY} docker build --pull -t "\${CI_REGISTRY_IMAGE}:\${IMAGE_TAG}" -f Dockerfile . </pre> sh "\${CI_REGISTRY_IMAGE}:\${IMAGE_TAG}"

<https://gitlab.ti.bfh.ch/cas-mio/ci-cd/automateallthings/gitlab-ci/-/ci/editor#>

Gitlab-Registry



- Aggregated View over Projects, Subprojects[0..*], Repositories
- Direct Access via Gitlab-CI
- Credential for pulling is needed
- If you want to mutate a project: clean the registry first

Documentation

- Gitlab Keyword Reference
- <https://docs.gitlab.com/ee/ci/pipelines/>
- <https://docs.gitlab.com/ee/ci/yaml/>
- <https://docs.gitlab.com/ee/ci/examples/>

Pipeline Libraries, Gitlab Components

Templates VS Components:

- Both used for referencing common steps shared between multiple pipelines
- Defined for given steps (might be able to be overridden)

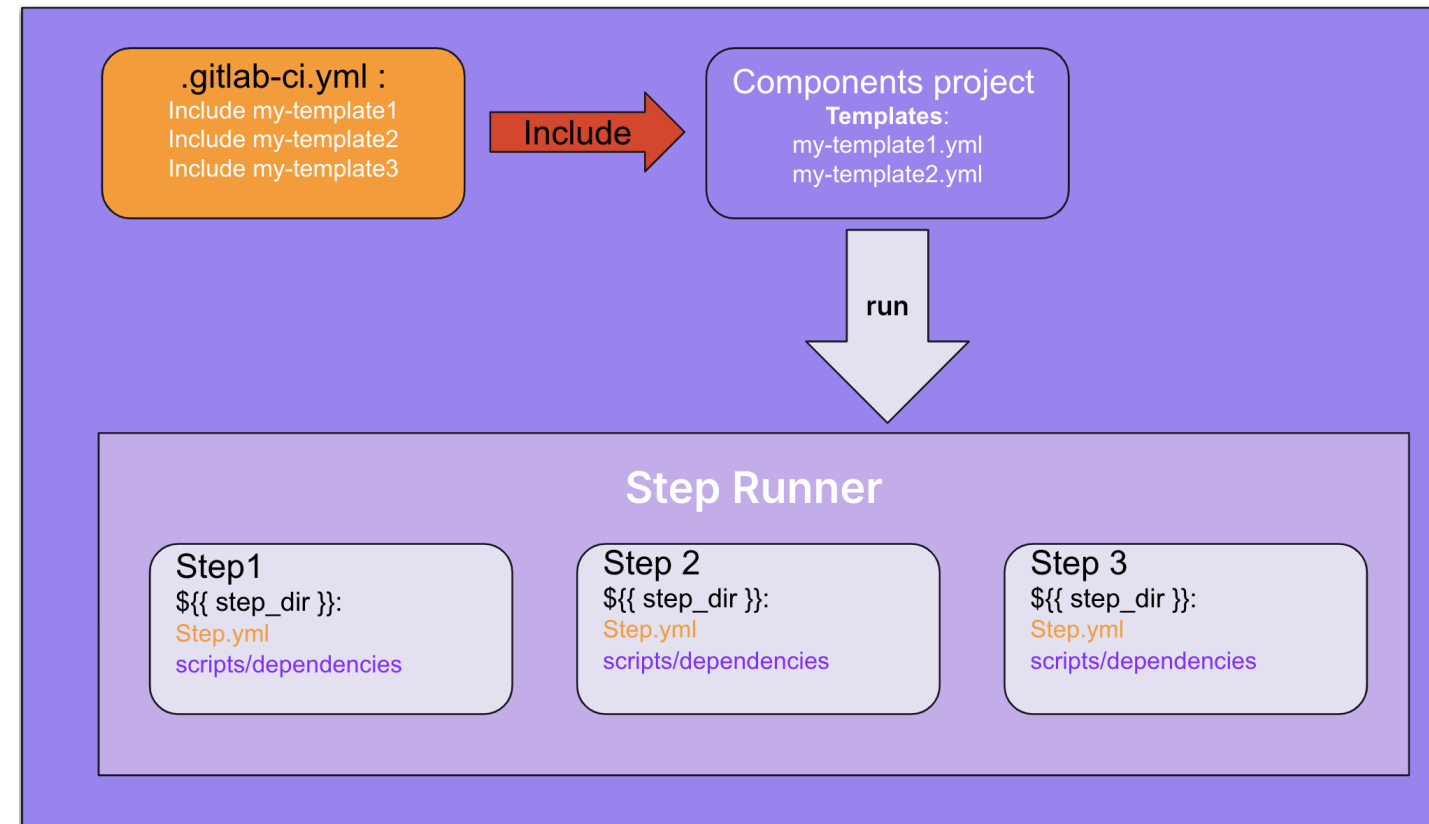
→ DRY-concept

Templates:

- Old but well integrated (several dashboards)
- Parametrization takes place via variables

Components:

- Newer concept with defined parameters
- [need to be mirrored](https://about.gitlab.com/blog/2024/10/16/how-to-include-file-references-in-your-ci-cd-components/) / included into components library (CI/CD catalogue)



<https://about.gitlab.com/blog/2024/10/16/how-to-include-file-references-in-your-ci-cd-components/>

Common Do's and don'ts

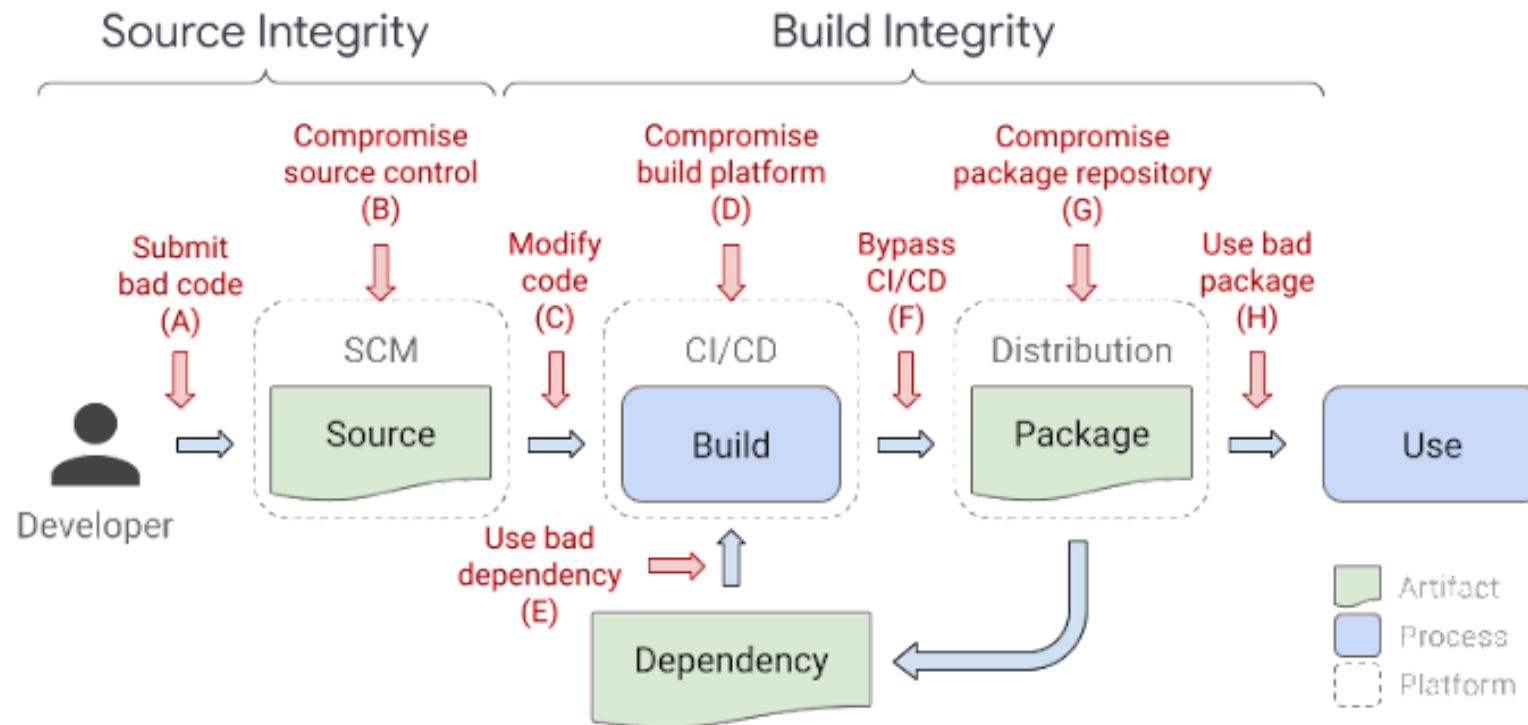
Do's

- Everything in sourcecode
- Do not reinvent the wheel
- Fail fast, if necessary
- Utilize entire internal toolset, variables methods, libs, ...
- Keep it simple
- When working with external services...
 - ...use time outs
 - make failure optional
- Sketch your build into steps...
 - Which build stage are you using for which step?
 - What can be executed in parallel?
 - Where are artifacts passed between the build steps?

Don'ts

- Beware of conflicts / intransparencies / lifecycle issues when using multiple libraries
- Do not expect everything works as expected
 - Debugging is exhausting
 - Reference literatur / documentation / sourcecode
- Tokens need to be rotated
 - Use your calendar
- Be aware of quotas in build regarding
 - Logs
 - Time
 - Number of Artifacts

Beyond Simple Building, SLSA



		Required at			
Requirement		SLSA 1	SLSA 2	SLSA 3	SLSA 4
Source	Version Controlled		✓	✓	✓
	Verified History			✓	✓
	Retained Indefinitely			18 mo.	✓
	Two-Person Reviewed				✓
Build	Scripted	✓	✓	✓	✓
	Build Service		✓	✓	✓
	Ephemeral Environment			✓	✓
	Isolated			✓	✓
	Parameterless				✓
	Hermetic				✓
	Reproducible				○
Provenance	Available	✓	✓	✓	✓
	Authenticated		✓	✓	✓
	Service Generated		✓	✓	✓
	Non-Falsifiable			✓	✓
	Dependencies Complete				✓
Common	Security				✓
	Access				✓
	Superusers				✓

<https://security.googleblog.com/2021/06/introducing-slsa-end-to-end-framework.html>

Beyond Simple Building: ChatOps, Dependency Mgmt, ...

Automatism extend simple building such as:

- Notifications
- Lifecycle Management of Packages
- Issue Management
- Pull Request Management
- ...



Name	Description	Install
auto-approve	Automatically approves and merges PRs matching user-specified configs	install
auto-label	Automatically labels issues and PRs with product, language, or directory based labels	install
blunderbuss	Assigns issues and PRs randomly to a specific list of users	install
cherry-pick-bot	Cherry-pick merged PRs between branches	install
conventional-commit-lint	PR checker that ensures that the commit messages follow conventionalcommits.org style	install
do-not-merge	PR checker that ensures the <code>do not merge</code> label is not present	install
failurechecker	Check for automation tasks, e.g., releases, that are in a failed state	install
flakybot	Listen on PubSub queue for broken builds, and open corresponding issues	install
generated-files-bot	PR checker to notify if you are modifying generated files	install
label-sync	Synchronize labels across organizations	install
license-header-lint	PR checker that ensures that source files contain valid license headers	install
merge-on-green	Merge a pull-request when all required checks have passed	install
policy	Check repo configuration against known rules	install
release-please	Proposes releases based on semantic version commits	install
release-trigger	Trigger releases jobs	install
repo-metadata-lint	Lint <code>.repo-metadata.json</code> files	install
snippet-bot	Check for mismatched region tags in PRs	install
sync-repo-settings	Synchronize repository settings from a centralized config	install
trusted-contribution	Allows Kokoro CI to trigger for trusted contributors	install

<https://github.com/googleapis/repo-automation-bots>

